

AutoLISP Builtins (Core)

User Manual

clautolisp contributors

June 24, 2026

Contents

1	Introduction	1
2	Loading	1
3	Installing the builtins	1
4	Adding a builtin	2
5	Example: writing a new pure-language builtin	2
6	Example: writing a HAL-mediated builtin	2
7	Querying the registry	2
8	See also	3
9	Synchronisation	3

1 Introduction

This manual is the programmer’s guide to `clautolisp.autolisp-builtins-core`. The audience is a Common Lisp developer adding a new AutoLISP built-in, replacing one, or driving the builtins layer from CL.

End-user AutoLISP developers don’t need this manual — they invoke the builtins from AutoLISP via their documented names. See the autolisp-spec reference for the surface contract.

2 Loading

```
(asdf:load-system "clautolisp/autolisp-builtins-core")
  Dependencies: clautolisp/autolisp-runtime + clautolisp/autolisp-host
+ clautolisp/autolisp-mock-host.
```

3 Installing the builtins

Call `install-core-builtins` once per session, after the runtime context is built:

```
(let ((context (clautolisp.autolisp-runtime:make-default-runtime-context
:dialect (clautolisp.autolisp-reader:autolisp-dialect-strict)))) (clautolisp.autolisp-
builtins-core:install-core-builtins) ;; ... run user code ... )
```

Every standard AutoLISP function name becomes a bound subr in the context's document namespace. Re-installing on the same context is idempotent — values are overwritten.

4 Adding a builtin

Three steps:

1. Write the CL implementation. Use `require-*` helpers to coerce / validate arguments. Signal `signal-builtin-argument-error` for type violations; raise `autolisp-runtime-error` for runtime issues. Return an AutoLISP value (autolisp-string, integer, real, list, autolisp-symbol, ...).

2. Register it via `make-core-builtin-subr`:

```
(make-core-builtin-subr "MY-FUNCTION" #'builtin-my-function)
```

The name is the uppercased AutoLISP symbol-name.

3. Add a test under `tests/` covering the happy path and the argument-validation path. Update `documentation/builtins-coverage.md` if the new builtin closes a gap.

5 Example: writing a new pure-language builtin

```
(defun builtin-double (arg) (let ((n (require-int-or-real arg "DOUBLE")))
(* 2 n)))
```

;; Registration goes alongside the other math entries: (make-core-builtin-subr "DOUBLE" #'builtin-double)

After `install-core-builtins` the AutoLISP user can call `(double 21)` and get 42.

6 Example: writing a HAL-mediated builtin

```
(defun builtin-my-hostop (arg) (let* ((host (current-evaluation-host))) (host-my-hostop host (require-string arg "MY-HOSTOP"))))  
  (make-core-builtin-subr "MY-HOSTOP" #'builtin-my-hostop)
```

The HAL generic function `host-my-hostop` is defined in `clautolisp.autolisp-host` alongside `NullHost`'s default `:host-not-supported` implementation. `MockHost` adds the in-memory test version.

7 Querying the registry

The runtime's `atoms-family` returns the bound symbol names — includes every registered builtin plus user-defined SETQ / DEFUN bindings.

;; In AutoLISP: (atoms-family 0) ; → list of symbols (atoms-family 1) ;
→ list of name strings

8 See also

- autolisp-builtins-core-specifications.org — internal design, categories, HAL contract.
- autolisp-spec/documentation/autolisp-visual-lisp-specification-draft.org — normative reference for every builtin.
- documentation/builtins-coverage.md — implementation status.
- `clautolisp.autolisp-host` / `clautolisp.autolisp-mock-host` — HAL surface the host-mediated builtins go through.

9 Synchronisation

Per `AGENTS.md`, code / spec / user-manual / coverage notes propagate. New host-mediated builtins also touch the HAL specs.